

Machine Learning for Intrusion Detection

Complete Line-by-Line Code Explanation

(Written so anyone can understand — no tech
background needed)

What is this project?

Imagine you have a security guard at the door of a building. This guard checks every person entering and decides — is this person **safe (normal)** or a **threat (attack)**?

This project builds that **digital security guard** using Machine Learning. It looks at network traffic (data flowing through the internet) and decides if it's normal or an attack.

STEP 1 — Gathering Our Tools

```
import pandas as pd
import numpy as np
from sklearn.ensemble import
RandomForestClassifier
from sklearn.model_selection import
train_test_split
from sklearn.preprocessing import
LabelEncoder
from sklearn.metrics import
classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
```

Think of this step like **unpacking a toolbox** before starting a job.

Each line brings in a specific tool we'll use later.

```
import pandas as pd
```

Real-world analogy: Imagine opening Microsoft Excel.

`pandas` is a tool that lets Python work with data in **rows and columns** — just like a spreadsheet.

- `import` = "bring this tool into our program"
 - `pandas` = the tool's full name
 - `as pd` = give it a nickname so we don't have to type "pandas" every time. We just type `pd`
-

```
import numpy as np
```

Real-world analogy: A super-fast calculator.

`numpy` is great at doing math on thousands of numbers at once – much faster than a regular Python loop.

- `as np` = short nickname for `numpy`
-

```
from sklearn.ensemble import
```

```
RandomForestClassifier
```

Real-world analogy: Hiring 100 experts and taking a vote.

`sklearn` (scikit-learn) is Python's most popular machine learning library.

`RandomForestClassifier` is the actual brain of our project – the algorithm that learns to detect attacks.

How does it work?

- It creates **100 decision trees** (like 100 judges)
 - Each judge looks at the data and gives their opinion: "normal" or "attack"
 - The **majority vote wins**
 - So if 87 out of 100 judges say "attack" → it's an attack
-

```
from sklearn.model_selection import  
train_test_split
```

Real-world analogy: A teacher separating exam practice papers from the real exam.

Before teaching our model, we split data into two parts:

- **Training data** (80%) → the model learns from this
- **Testing data** (20%) → we use this to check if the model actually learned

We never show testing data during training. Otherwise it's like a student memorizing the answer sheet — not real learning.

```
from sklearn.preprocessing import  
LabelEncoder
```

Real-world analogy: A translator converting words to numbers.

Computers and ML models **only understand numbers**. They can't process words like "normal" or "attack".

`LabelEncoder` converts text → numbers:

- `normal` → `0`
- `attack` → `1`

- `tcp` → 0, `udp` → 1, `icmp` → 2
-

```
from sklearn.metrics import  
classification_report, confusion_matrix
```

Real-world analogy: A report card after an exam.

After our model makes predictions, we need to check how well it did.

- `classification_report` → shows accuracy, precision, and recall as percentages
 - `confusion_matrix` → shows a grid of correct vs. incorrect predictions
-

```
import matplotlib.pyplot as plt
```

Real-world analogy: A drawing tool.

`matplotlib` lets Python draw **graphs, charts, and plots**.

We use it to display our confusion matrix visually.

```
import seaborn as sns
```

Real-world analogy: A prettier drawing tool.

`seaborn` is built on top of `matplotlib` and makes graphs look more **beautiful and colorful** with less code.



STEP 2 – Loading the Dataset

```
url =  
"https://raw.githubusercontent.com/defcom17/  
NSL_KDD/master/KDDTrain+.txt"
```

Real-world analogy: Opening a file from Google Drive via a link.

Instead of uploading a file manually, we paste a **direct link** to the dataset.

The dataset lives on GitHub (a website for storing code and data files).

Python will download it automatically when we run this line.

```
columns = [  
    "duration", "protocol_type", "service",  
    "flag", "src_bytes", "dst_bytes",  
    "land", "wrong_fragment", "urgent",  
    "hot", "num_failed_logins", "logged_in",  
    "num_compromised", "root_shell",
```

```

"su_attempted", "num_root",
"num_file_creations",
    "num_shells", "num_access_files",
"num_outbound_cmds", "is_host_login",
    "is_guest_login", "count", "srv_count",
"error_rate", "srv_error_rate",
    "error_rate", "srv_error_rate",
"same_srv_rate", "diff_srv_rate",
    "srv_diff_host_rate", "dst_host_count",
"dst_host_srv_count",
    "dst_host_same_srv_rate",
"dst_host_diff_srv_rate",
"dst_host_same_src_port_rate",
    "dst_host_srv_diff_host_rate",
"dst_host_error_rate",
"dst_host_srv_error_rate",
    "dst_host_error_rate",
"dst_host_srv_error_rate", "label",
"difficulty"
]

```

Real-world analogy: Adding headers to a spreadsheet that has no headers.

The dataset file is just raw data — numbers and text with **no column titles**.

This list tells pandas what to call each column.

Think of it like receiving a table like this:

```

tcp    http    SF      181     5450    0      0 ...

```

...and adding the header row yourself:

```
protocol_type | service | flag | src_bytes  
| dst_bytes | land | wrong_fragment ...
```

Important columns explained simply:

Column	What it means in plain English
<code>duration</code>	How long was this internet connection? (in seconds)
<code>protocol_type</code>	What type of connection? (TCP, UDP, or ICMP)
<code>src_bytes</code>	How much data was sent by the user?
<code>dst_bytes</code>	How much data was received back?
<code>label</code>	Is this connection normal or an attack ? ← This is what we predict

```
df = pd.read_csv(url, header=None,  
names=columns)
```

Breaking this line down word by word:

Part	Meaning
<code>df</code>	Short for "dataframe" — a variable that stores our table of data
<code>pd.read_csv(...)</code>	Use pandas to read a CSV file (comma-separated data)
<code>url</code>	The internet link we defined above
<code>header=None</code>	Tell pandas: "this file has NO header row, don't read the first row as headers"
<code>names=columns</code>	Instead, use our <code>columns</code> list as the headers

Real-world analogy: Opening a CSV file in Excel and manually typing in the column names since the file came without them.

```
df.drop("difficulty", axis=1, inplace=True)
```

Part	Meaning
<code>df.drop(...)</code>	Delete something from our table
<code>"difficulty"</code>	The column we want to delete
<code>axis=1</code>	We're deleting a column (axis=0 would delete a row)
<code>inplace=True</code>	Apply the change to the original data, don't create a copy

Real-world analogy: Deleting a column in Excel that you don't need.

We don't need the "difficulty" column for our prediction model — it just rates how hard each sample is to classify. So we remove it to keep things clean.

```
print(df.shape)  # rows and columns count
print(df.head()) # first 5 rows
```

- `df.shape` → prints something like `(125973, 42)` meaning **125,973 rows and 42 columns**
- `df.head()` → shows the **first 5 rows** so you can visually check the data looks right

Real-world analogy: Glancing at the top of a spreadsheet to make sure it loaded correctly.

STEP 3 — Exploring Labels

```
print(df['label'].value_counts())
```

Part	Meaning
<code>df['label']</code>	Access the "label" column of our table
<code>.value_counts()</code>	Count how many times each unique value appears

Real-world analogy: Going through a stack of medical test results and counting — how many say "positive" and how many say "negative"?

This tells us: out of ~125,000 network connections, how many are normal and how many are attacks? And what *type* of attacks are there?

The output shows 21+ different attack types like: `neptune`, `smurf`, `back`, `teardrop` etc.

STEP 4 — Simplifying Labels

```
df['label'] = df['label'].apply(lambda x:  
    'normal' if x == 'normal' else 'attack')
```

This is one line doing a big job. Let's break it apart:

The problem: Our data has 21 different attack types. For a beginner model, we just need 2 categories: normal or attack.

The solution: Go through every row and simplify.

```
df['label']
```

Access the label column. Like clicking on a column in Excel.

```
.apply(...)
```

Go through **every single row** in that column and run a function on it.

Real-world analogy: Imagine 125,000 sticky notes each with a label on them. `.apply()` picks up each note one by one and does something with it.

```
lambda x:
```

This creates a **mini one-line function**. `x` = the current value being processed.

Real-world analogy: Think of it as: *"For each sticky note I pick up, call its value `x`, then..."*

```
'normal' if x == 'normal' else 'attack'
```

Plain English translation:

"If the value on this note says 'normal' → keep it as 'normal'. If it says ANYTHING ELSE (neptune, smurf, back, etc.) → change it to 'attack'."

So after this step:

- `normal` → stays `normal`
 - `neptune` → becomes `attack`
 - `smurf` → becomes `attack`
 - `back` → becomes `attack`
 - *(all 20+ attack types)* → all become `attack`
-

```
print(df['label'].value_counts())
```

Check the result — now we should only see two values: `normal` and `attack`.

1234 STEP 5 — Converting Text to Numbers

```
le = LabelEncoder()
```

Real-world analogy: Taking a translation dictionary off the shelf and placing it on your desk.

We create the `LabelEncoder` tool and store it in a variable called `le`.

We haven't used it yet — just prepared it.

```
for col in ['protocol_type', 'service',
            'flag', 'label']:
    df[col] = le.fit_transform(df[col])
```

```
for col in [...]:
```

This is a **loop** — it repeats the next line for each item in the list.

The list has 4 items, so the loop runs 4 times:

1. First time → `col = 'protocol_type'`
2. Second time → `col = 'service'`
3. Third time → `col = 'flag'`

4. Fourth time → `col = 'label'`

Real-world analogy: You have 4 columns with words. You go to each one and swap all the words for numbers.

`le.fit_transform(df[col])`

Two things happen at once here:

Step	What it does	Example
<code>fit</code>	Looks at all unique values in the column	Sees: <code>tcp</code> , <code>udp</code> , <code>icmp</code>
<code>transform</code>	Converts each value to a number	<code>icmp</code> → 0, <code>tcp</code> → 1, <code>udp</code> → 2

`df[col] = ...`

Replace the original text column with the new number column.

Before:

```
protocol_type
tcp
```

```
udp
tcp
icmp
```

After:

```
protocol_type
1
2
1
0
```

```
print(df.head())
```

Check the first 5 rows again — now all columns should contain only numbers.

STEP 6 — Splitting Data into Training & Testing

```
X = df.drop('label', axis=1)
y = df['label']
```

X = Features (the inputs)

Everything except the label column.

These are all the clues the model gets to make its decision:

- How long was the connection?
- How much data was sent?
- What protocol was used?
- etc.

Real-world analogy: The description of a suspicious person – height, clothing, behaviour.

y = Label (the answer)

Just the label column – either 0 (normal) or 1 (attack).

Real-world analogy: The actual verdict – "was this person a criminal or not?"

The model learns to map $\mathbf{X} \rightarrow \mathbf{y}$. Given the features, predict the label.

```
X_train, X_test, y_train, y_test =  
train_test_split(X, y, test_size=0.2,  
random_state=42)
```

This one line creates **4 separate boxes** of data:

Variable	What it contains
<code>X_train</code>	80% of the features → used to teach the model
<code>X_test</code>	20% of the features → used to test the model
<code>y_train</code>	The correct answers for the training data
<code>y_test</code>	The correct answers for the testing data (kept hidden during training)

`test_size=0.2`

Use 20% for testing, 80% for training.

`random_state=42`

This makes the split **reproducible**. Without it, you'd get a different random split every time. `42` is just a conventional number — you could use any number.

Real-world analogy: A teacher photocopies 100 exam papers. 80 copies go to students to practice. 20 copies are locked away as the real exam.

```
print("Training size:", X_train.shape)
```

```
print("Testing size:", X_test.shape)
```

Prints how many rows are in each set so you can confirm the split worked.

STEP 7 – Training the Model

```
rf_model =  
RandomForestClassifier(n_estimators=100,  
random_state=42)
```

RandomForestClassifier(n_estimators=100, ...)

Creates the model with settings:

Setting	Meaning
<code>n_estimators=100</code>	Build 100 decision trees (100 "judges" who each vote)
<code>random_state=42</code>	Makes results reproducible – same result every run

Real-world analogy: Hiring 100 security experts and briefing them – "you'll each review network connections and vote on whether it's an attack."

```
rf_model.fit(X_train, y_train)
```

```
rf_model.fit(X_train, y_train)
```

This is where actual learning happens.

`fit` = train the model.

We pass in:

- `X_train` = the features (all the clues)
- `y_train` = the correct answers

The model studies thousands of examples:

```
"When src_bytes = 0 and protocol = icmp and  
service = ecr_i → that was an attack."  
"When duration = 0 and src_bytes = 181 and  
dst_bytes = 5450 → that was normal."
```

It learns patterns like a student memorizing what
"attack traffic" looks like versus "normal traffic".

Real-world analogy: Showing those 100 experts
thousands of past cases — each labeled "normal" or
"attack" — and letting them study the patterns.

```
print("Model training complete!")
```

Just a message to tell you training is done. Training can take a few seconds to minutes depending on dataset size.



STEP 8 – Testing the Model

```
y_pred = rf_model.predict(X_test)
```

```
rf_model.predict(X_test)
```

Now we give the model the **test data** (the 20% it has never seen before) and ask it: "what do you think these are?"

- `X_test` = the features of the test data (no labels – the model can't see the answers)
- `y_pred` = the model's guesses (predictions)

Real-world analogy: Those 100 security experts now review 100 NEW cases they've never seen. They vote on each one: "normal or attack?"

```
print(classification_report(y_test,  
y_pred))
```

```
classification_report(y_test, y_pred)
```

Compares:

- `y_test` = the real correct answers
- `y_pred` = the model's predicted answers

Then prints a report like a scorecard.

Sample Output:

	precision	recall	f1-score
support			
attack	1.00	1.00	1.00
13422			
normal	1.00	1.00	1.00
11773			
accuracy			1.00
25195			

Term	Plain English
Precision	Of all the things the model called "attack", how many were actually attacks?
Recall	Of all the real attacks in the data, how many did the model catch?

Term	Plain English
F1-score	A balance score combining precision and recall
Accuracy	Overall – what percentage of all predictions were correct?

STEP 9 – Visualising the Results (Confusion Matrix)

```
cm = confusion_matrix(y_test, y_pred)
```

Creates a **2×2 grid** comparing actual labels vs predicted labels.

Real-world analogy: A scoreboard comparing what actually happened vs what the security guard said.

```
plt.figure(figsize=(6, 4))
```

Sets up a blank canvas for our chart.

`figsize=(6, 4)` = 6 inches wide, 4 inches tall.

```
sns.heatmap(cm, annot=True, fmt='d',
             cmap='Blues',
             xticklabels=['Normal',
                          'Attack'],
             yticklabels=['Normal',
                          'Attack'])
```

Draws the heatmap (coloured grid) using seaborn:

Part	Meaning
<code>cm</code>	The confusion matrix data we just created
<code>annot=True</code>	Show the actual numbers inside each box
<code>fmt='d'</code>	Display numbers as whole integers (not 1.0, just 1)
<code>cmap='Blues'</code>	Use a blue colour scheme (darker = higher number)
<code>xticklabels</code>	Labels on the X-axis (bottom): Normal, Attack
<code>yticklabels</code>	Labels on the Y-axis (side): Normal, Attack

```
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Intrusion
Detection')
plt.show()
```

- `xlabel` = label for the horizontal axis
- `ylabel` = label for the vertical axis
- `title` = heading for the chart
- `plt.show()` = display the chart

Reading the Confusion Matrix

	Predicted: Normal
Predicted: Attack	
Actual: Normal	11,746
27	
Actual: Attack	7
13,415	

Box 1 – Top Left: 11,746 

True Negative

- The model said: "Normal"
- Reality: It WAS normal
-  Correct – 11,746 safe connections correctly identified

Box 2 – Top Right: 27 ❌

False Positive (False Alarm)

- The model said: "Attack"
- Reality: It was actually NORMAL traffic
- ❌ Wrong – 27 innocent connections wrongly flagged
- **Real-world analogy:** Security guard stops an innocent person

Box 3 – Bottom Left: 7 ❌

False Negative (MOST DANGEROUS)

- The model said: "Normal"
- Reality: It was actually an ATTACK
- ❌ Wrong – 7 real attacks slipped through undetected
- **Real-world analogy:** Security guard lets a criminal walk in

Box 4 – Bottom Right: 13,415 ✅

True Positive

- The model said: "Attack"
 - Reality: It WAS an attack
 - ✅ Correct – 13,415 attacks successfully caught
-

Final Score

Metric	Value
Total predictions	25,195
Correct predictions	25,161
Wrong predictions	34
Overall Accuracy	~99.86%

The model missed only **7 attacks** out of 13,422. That's exceptional performance.

Complete Flow Summary

```
Raw Dataset (internet link)
  ↓
Load & name columns (pandas)
  ↓
Simplify labels (21 attack types → just
"attack")
  ↓
Convert text to numbers (LabelEncoder)
  ↓
Split: 80% Train / 20% Test
  ↓
Train Random Forest (100 decision trees)
  ↓
```

Predict on test data



Evaluate with classification report +
confusion matrix



✅ Done – 99.86% accurate intrusion
detection!

*Written for beginners. Every line explained. No prior
coding knowledge assumed.*